Problem Set – Pass by Reference Functions. Create an IPO for each problem within this document. Save the document and upload to Blackboard for grading. Then create a C++ program for each problem. Upload the .CPP files to Blackboard for grading.

1. Allow the user to enter a quantity and price, use ctl+z to stop. Use one function to compute the total (quantity times price), tax (7% of total) and total order (total plus tax). The function should be passed the quantity and price by value and total, tax and total order by reference. Display total, tax and total order in main. Sum and display total of all orders and tax for all orders and display after the loop (all data is processed).

<table>
<tr><th>Input</th><th>Process</th><th>Output</th></tr>
<tr><td rowspan="2">• Quantity<br>• Price</td><td>computeTotal(Quantity & Price [Ref: Subtotal, Tax, & Total])<br>• Calculate Subtotal by finding the product of Quantity and Price<br>• Calculate Tax by finding the product of Subtotal and 7% (0.07)<br>• Calculate Total by finding the sum of Subtotal and Tax</td><td>Each Entry<br>• Subtotal<br>• Tax<br>• Total</td></tr>
<tr><td>Main()<br>• Declare Quantity, Price, Subtotal, Tax, & Total<br>• Declare Subtotal Sum & Tax Sum, set at 0<br>• Assign the input for Quantity and Price as their variables<br>• While not EOF<br>• Call computeTotal<br>• Add Subtotal to Subtotal Sum<br>• Add Tax to Tax Sum<br>• Display Subtotal, Tax, & Total<br>• Ask for new entry for Quantity and Price<br>• End Loop<br>• Display Subtotal Sum & Tax Sum</td><td>After All Entries<br>• Subtotal Sum<br>• Tax Sum</td></tr>
</table>

2. Enter the weight of a package and zip code. Use ctl+z to stop. Use a single function to do the computations specified next. Pass these weight and zip code by value, and pass postage, area charge and weight charge by reference. Compute postage to be sum of weight charge and area charge. Use tables below to find the charges. Compute weight charge to be weight x weight charge per ounce. Find the area charge in the table based on zip code. Then compute postage to be area charge plus weight charge. The function should return the weight charge, area charge and postage. Display area charge, weight charge and postage. Count and display the number of entries made.

**Area Table – Used to determine the area charge**

| Area | Area Charge |
|---|---|
| 60171 | $2.00 |
| 60172 | $2.50 |
| 60635 | $3.00 |
| All others | $5.00 |

**Weight Table – used to determine the weight charge**

| Weight | Weight Charge per Ounce |
|---|---|
| >100 | 0.02 |
| >50 | 0.03 |
| All other | 0.05 |

| Input | Process | Output |
|---|---|---|
| • Package Weight<br>• Zip Code | *computePackageCost(Package Weight & Zip Code [Ref: Area charge, Weight charge & Postage])*<br>• If Zip Code equals 60171, set Area Charge at $2.00<br>  ○ If Zip Code equals 60172, set Area Charge at $2.50<br>  ○ If Zip Code equals 60635, set Area Charge at $3.00<br>  ○ Otherwise, set Area Charge at $5.00<br>• If Package Weight greater than 100, calculate Weight Charge by finding the product Package Weight and 0.02 | *Each Entry*<br>• Area Charge<br>• Weight Charge<br>• Postage |

| | | |
|---|---|---|
| | ○ If Package Weight greater than 50, calculate Weight Charge by finding the product Package Weight and 0.03<br>○ Otherwise, calculate Weight Charge by finding the product Package Weight and 0.05<br>• Calculate Postage by finding the sum of Area Charge & Weight Charge | |
| | *Main()*<br>• Declare Package Weight, Zip Code, Area Charge, Weight Charge, & Postage<br>• Declare Number of Packages, set at 0<br>• Assign the input for Package Weight and Zip Code as their variables<br>• **While not EOF**<br>• Call *computePackageCost*<br>• Add 1 to Number of Packages<br>• Display Area Charge, Weight Charge, & Postage<br>• Ask for new entry for Package Weight and Zip Code<br>• **End Loop**<br>• Display Number of Packages | *For All Entries*<br>• Number of Packages |

3. Enter the student's last name, credit hours and financial aid, use ctl+z to stop. Pass credit hours and financial aid to a function by value. Pass tuition and tuition owed by reference. Compute tuition to be credit hours times $250. Compute tuition owed to be tuition minus the financial aid. Display student's last name, tuition and tuition owed. Sum and display total tuition owed by all students, count of number of entries and average amount owed by students.

| Input | Process | Output |
|---|---|---|
| • Student's Last Name<br>• Credit Hours<br>• Financial Aid | *computeTuition(Credit Hours & Financial Aid [Ref: Tuition & Tuition Owed])*<br>• Calculate Tuition by finding the product of Credit Hours and 250<br>• Calculate Tuition Owed by finding the difference between Tuition and Financial Aid | *Each Entry*<br>• Student's Last Name<br>• Tuition<br>• Tuition Owed |
| | *Main()*<br>• Declare Student's Last Name, Credit Hours, Financial Aid, Tuition, & Tuition Owed<br>• Declare Total of all Student's Tuition Owed, Number of Students, & Average Tuition Owed by Students, set at 0 | *After All Entries*<br>• Total of all students' Tuition Owed<br>• Number of Students |

| | | |
|---|---|---|
| | • Assign the input for Student's Last Name, Credit Hours, & Financial Aid as their variables<br>• **While not EOF**<br>• Call *computeTuition*<br>• Add Tuition Owed to Total of all students' Tuition Owed<br>• Add 1 to Number of Students<br>• Add Tuition Owed to Average Tuition Owed by Students<br>• Display Student's Last Name, Tuition, & Tuition Owed<br>• Ask for new entry for Student's Last Name, Credit Hours, & Financial Aid<br>• **End Loop**<br>• Calculate Average Tuition Owed by Students by finding the Quotient of Total of all students' Tuition Owed and Number of Students<br>• Display Total of all students' Tuition Owed, Number of Students, & Average Tuition Owed by Students | • Average Tuition Owed by Students |

4. Enter a number of widgets, use ctl+z to stop. Pass the number to a function by value, use ctl+z to stop. Use a single function to determine the cost per widget using the cost table below. Then compute extended price (number of widgets x cost per widget) and 7% sales tax. Finally compute total order to be extended price plus sales tax. Pass cost per widget, extended price, sales tax and total order by reference. For each line, display number of widgets, cost per widget, extended price, sales tax and total order. Sum all total orders and display when there is no more data to process.

**Cost Table**

| Number of Widgets | Cost Per Widget |
|---|---|
| 10000 and up | 4.00 |
| 5,000 and up | 5.00 |
| All other amounts | 10.00 |

| Input | Process | Output |
|---|---|---|
| • Number of Widgets | *computeWidgetOrder(Number of Widgets [Ref: Cost per Widget, Extended Price, Sales Tax Amount and Total Order Amount])* | *Each Entry*<br>• Number of Widgets<br>• Cost per Widget<br>• Extended Price<br>• Sales Tax Amount |

| | | |
|---|---|---|
| | • If Number of Widgets is greater than or equal to 10000, set Cost per Widget at $4.00<br>○ If Number of Widgets is greater than or equal to 5000, set Cost per Widget at $5.00<br>○ Otherwise, set Cost per Widget at $10.00<br>• Calculate Extended Price by finding the product of Number of Widgets and Cost per Widget<br>• Calculate Sales Tax Amount by finding the product of Extended Price and 7% (0.07)<br>• Calculate Total Order Amount by finding the sum of Extended Price and Sales Tax Amount | • Total Order Amount |
| | *Main()*<br>• Declare Number of Widgets, Cost per Widget, Extended Price, Sales Tax Amount, & Total Order Amount<br>• Declare Sum of all Total Order Amount, set at 0<br>• Assign the input for Number of Widgets as their variables<br>• **While not EOF**<br>• Call *computeWidgetOrder*<br>• Add Total Order Amount to Sum of all Total Order Amount<br>• Display Number of Widgets, Cost per Widget, Extended Price, Sales Tax Amount, & Total Order Amount<br>• Ask for new entry for Number of Widgets<br>• **End Loop**<br>• Display Sum of all Total Order Amount | *After All Entries*<br>• Sum of all Total Order Amount |

5. Enter the amount of investment, the 5 year interest rate and 10 year interest rate, use ctl+z to stop. Pass the amount and interest rates to a function by value. Pass variables representing five year amount and ten year amount to the same function by reference. Compute the five year amount and ten year amount using the formula below. Display the amount of the investment, the five year amount and the ten year amount.

   Five year amount = amount of the investment x (1 + 5 year rate) raised to $5^{th}$ power.

   Ten year amount = amount of the investment x (1+10 year rate) raised to $10^{th}$ power.

   Note: Enter the 5 year rate and 10 year rate in decimal form, i,e 5% is entered as 0.05.

Also Note: you need to use the pow built in function. Recall the pow function syntax:

pow (base, exponent)

In this case, base is 1 + 5 year rate and exponent is 5.

Another line will be: base is 1 + 10 year rate and exponent is 10.

Use #include<math.h> for the pow function.

<table>
<tr><th>Input</th><th>Process</th><th>Output</th></tr>
<tr><td rowspan="2">• Amount Invested<br>• 5-Year Interest Rate<br>• 10-Year Interest Rate</td><td>computeInvestment(Amount Invested, 5-Year Interest Rate, & 10-Year Interest Rate [Ref: Amount in 5-Years & Amount in 10-Years])<br>• Calculate Amount in 5-Years by finding the product of Amount Invested and (1 + 5-Year Interest Rate) ^5<br>• Calculate Amount in 10-Years by finding the product of Amount Invested and (1 + 10-Year Interest Rate) ^10</td><td rowspan="2">Each Entry<br>• Amount Invested<br>• Amount in 5-Years<br>• Amount in 10-Years</td></tr>
<tr><td><u>Main()</u><br>• Declare Amount Invested, 5-Year Interest Rate, 10-Year Interest Rate, Amount in 5-Years, & Amount in 10-Years<br>• Assign the input for Amount Invested, 5-Year Interest Rate, & 10-Year Interest Rate<br>• While not EOF<br>• <u>Call computeInvestment</u><br>• Display Amount Invested, Amount in 5-Years, & Amount in 10-Years<br>• Ask for new entry for Amount Invested, 5-Year Interest Rate, & 10-Year Interest Rate<br>• End Loop</td></tr>
</table>